FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Diaz, James Daniel M.
Section:	AN42
Professor:	Crisola G. Tan
Date:	October 5, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA4)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why? RBR was easier to implement because it only involved simple if-then statements to enforce rules, as opposed to CBR which leans in a more dynamic direction based on similarity score.
2. Which is more flexible in solving unstructured problems? CBR is probably more flexible in solving unstructured problems because it doesn’t need as much concrete info as RBR does.
3. Which is better for situations with continuously changing data? CBR is better for situations with continuously changing data.
4. Examples of real-world applications for both. CBR can be used for medical treatment, RBR can be used for equipment diagnosis.

Your Code - Laboratory Tasks

```
# -*- coding: utf-8 -*-
"""TA4_Diaz.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1EOBvT41UOnO-CKV92rx1goEmDI5QqIbv
"""

# Knowledge Base
knowledge_base = {
    "Laptop": {
        "Physical": {
            "Color": "Black",
            "Weight": 1.5
        },
        "Technical": {
            "Processor": "Intel Core i5",
            "RAM": 16
        },
        "Usage": {
            "Purpose": "School and work"
        }
    }
}

# Inference Engine
def inference_engine(kb):
    laptop = kb["Laptop"]

    # Rule 1
    if laptop["Technical"]["RAM"] >= 8:
        print("Action: Laptop is suitable for multitasking.")

    # Rule 2
    if laptop["Technical"]["Processor"] == "Intel Core i5":
        print("Action: Laptop is suitable for school and work.")

    # Rule 3
    if laptop["Physical"]["Weight"] <= 2:
        print("Action: Laptop is portable.")
```

```
    # Rule 4
    if laptop["Physical"]["Color"] == "Black":
        print("Action: Laptop has a black exterior.")
```

```
    # Rule 5
    if laptop["Usage"]["Purpose"] == "School and work":
        print("Action: Laptop can be used for productivity tasks.")
```

```
# Run the inference engine
inference_engine(knowledge_base)
```

```
# Case-Based Reasoning Knowledge Base
```

```
case_base = {
    "Case 1": {
        "problem_features": {
            "device": "Laptop",
            "problem": "Overheating",
            "symptom": "Fan is loud"
        },
        "solution": "Clean the cooling fan and vents."
    },
```

```
    "Case 2": {
        "problem_features": {
            "device": "Laptop",
            "problem": "Slow performance",
            "symptom": "Programs take a long time to open"
        },
        "solution": "Close unnecessary programs and upgrade the RAM."
    },
```

```
    "Case 3": {
        "problem_features": {
            "device": "Laptop",
            "problem": "Battery draining quickly",
            "symptom": "Battery lasts less than 2 hours"
        },
        "solution": "Reduce power usage and replace the battery if necessary."
    }
}
```

```
# --------------------------------
# 1. RETRIEVE
# --------------------------------

def retrieve_case(new_problem):
    best_case = None
    highest_similarity = 0

    for case_name, case in case_base.items():
        similarity = 0

        for feature in new_problem:
            if feature in case["problem_features"]:
                if new_problem[feature] == case["problem_features"][feature]:
                    similarity += 1

        if similarity > highest_similarity:
            highest_similarity = similarity
            best_case = case_name

    return best_case


# --------------------------------
# 2. REUSE
# --------------------------------

def reuse_solution(case_name):
    return case_base[case_name]["solution"]


# --------------------------------
# 3. REVISE
# --------------------------------

def revise_solution(solution, new_problem):
    print("\nOriginal solution:", solution)

    # Example revision based on the new problem
    if new_problem["problem"] == "Overheating":
        solution = "Clean the cooling fan and vents, and make sure the laptop has
proper airflow."

    print("Revised solution:", solution)
```

```
    return solution
```

```
# --------------------------------
# 4. RETAIN
# --------------------------------
```

```
def retain_case(new_problem, solution):
    new_case_number = len(case_base) + 1
    new_case_name = "Case " + str(new_case_number)
```

```
    case_base[new_case_name] = {
        "problem_features": new_problem,
        "solution": solution
    }
```

```
    print("\nNew case retained as:", new_case_name)
```

```
# --------------------------------
# NEW PROBLEM
# --------------------------------
```

```
new_problem = {
    "device": "Laptop",
    "problem": "Overheating",
    "symptom": "Fan is loud"
}
```

```
# CBR PROCESS
```

```
print("New Problem:", new_problem)
```

```
# Retrieve
retrieved_case = retrieve_case(new_problem)
print("\nRetrieved:", retrieved_case)
```

```
# Reuse
solution = reuse_solution(retrieved_case)
print("Reused Solution:", solution)
```

```
# Revise
solution = revise_solution(solution, new_problem)
```

```
# Retain
```

```
retain_case(new_problem, solution)
```

```
# Display updated knowledge base
print("\nUpdated Case Base:")
for case, details in case_base.items():
    print(case, ":", details)
```